Applied ML HW2 Report

Charlotte Zhang

1. **Introduction** **Pg 2**
2. **Data Preparation** **Pg 2-3**
    - 2.1 Dataset Overview
    - 2.2 Data Cleaning
    - 2.3 Train, Validation, and Test Splits
    - 2.4 Feature Preprocessing
3. **Gradient Boosted Decision Trees** **Pg 3-4**
    - 3.1 Model and Experimental Design
    - 3.2 Hyperparameter Results
    - 3.3 Training Behavior and Feature Importance
4. **Multi-Layer Perceptron** **Pg 4-5**
    - 4.1 Model and Experimental Design
    - 4.2 Hyperparameter Results
    - 4.3 Convergence and Training Loss
5. **Final Model Evaluation and Comparison** **Pg 6-7**
    - 5.1 Model Selection
    - 5.2 Test Performance
    - 5.3 Precision-Recall Curves and Confusion Matrices
    - 5.4 Computational Efficiency
    - 5.5 Discussion and Conclusion
    - 5.6 AI Tool Disclosure

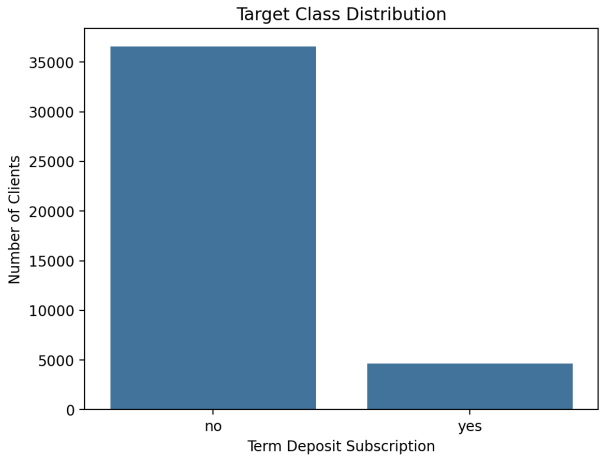
1. Introduction

The objective of this experiment was to compare Gradient Boosted Decision Trees (GBDT) and a Multi-Layer Perceptron (MLP) on the Bank Marketing dataset. The goal was to evaluate their predictive performance, training behavior, computational efficiency, and sensitivity to hyperparameters. The prediction task was to determine whether a client would subscribe to a term deposit (yes or no) based on demographic, financial, and campaign-related information.

2. Data Preparation

2.1 Dataset Overview

The Bank Marketing dataset contained 41,188 observations and 21 original columns, including 20 predictors and the binary target variable y. The target variable was imbalanced: 36,548 clients (88.73%) did not subscribe to a term deposit; 4,640 clients (11.27%) subscribed to a term deposit. The positive class was defined as yes. This imbalance makes accuracy alone insufficient for evaluating model performance, so Average Precision was used as the primary evaluation metric.



2.2 Data Cleaning

First, the duration variable was removed because it represents the duration of the last contact with the client to avoid data leakage. Second, the categorical value unknown was kept as a separate category rather than treated as missing. This preserves information about unavailable or unreported client characteristics. Third, the special value pdays = 999 was treated as an indicator that a client had not previously been contacted. A binary feature, previously_contacted, was created to preserve this information, and the value 999 was replaced with missing data. Missing numerical values were subsequently imputed using the training-set median. After cleaning, the dataset contained 20 predictors: 10 numerical and 10 categorical features.

2.3 Train, Validation, and Test Splits

The data were divided into training (70%), validation (15%), and test (15%) sets using stratified sampling with a random seed of 42. Stratification maintained approximately the same class distribution across all three sets.

	Dataset string	Rows int64	Positive Class Pr... string
0	Overall	41188	11.2654%
1	Training	28831	11.2657%
2	Validation	6178	11.2658%
3	Test	6179	11.2640%

2.4 Feature Preprocessing

Numerical features were median-imputed, while categorical features were one-hot encoded. For the MLP, numerical features were additionally standardized using StandardScaler, since neural networks are sensitive to differences in feature scales. Standardization was not applied to the GBDT because decision trees generally do not require scaled inputs. All preprocessing steps were fitted using only the training data and then applied to the validation and test sets. Both models received the same underlying information and used identical data splits. After preprocessing, both models had 63 input features.

3. Gradient Boosted Decision Trees

3.1 Model and Experimental Design

GBDT was implemented using XGBClassifier with the histogram-based tree construction method. The maximum number of estimators was set to 3,000, with early stopping after 30 consecutive rounds without improvement in validation log loss. Training and validation log loss were monitored during fitting, with the validation set used to control early stopping. Three groups of hyperparameter experiments were performed, changing one parameter at a time while holding the others fixed:

- Learning rate: 0.01, 0.1, and 0.3.
- Maximum tree depth: 2, 3 (baseline), 4, and 6.
- Subsample: 0.6, 0.8 (baseline), and 1.0.

The baseline configuration was reused rather than redundantly trained in multiple experiment groups, resulting in eight distinct configurations. Average Precision on the validation set was used to select the best configuration.

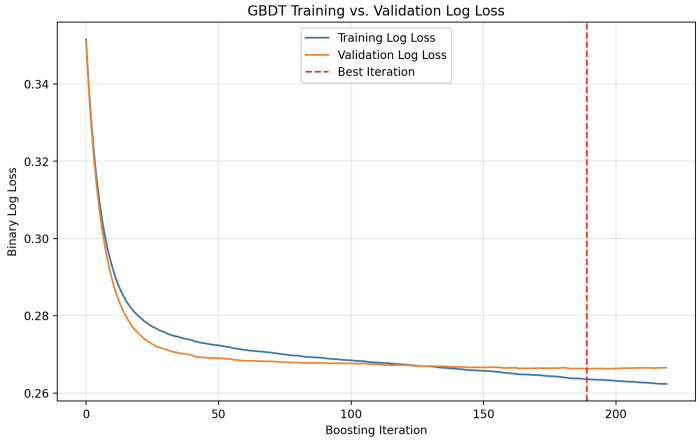
3.2 Hyperparameter Results

The best-performing GBDT configuration used a learning rate of 0.1, maximum depth of 3, and subsample of 1.0.

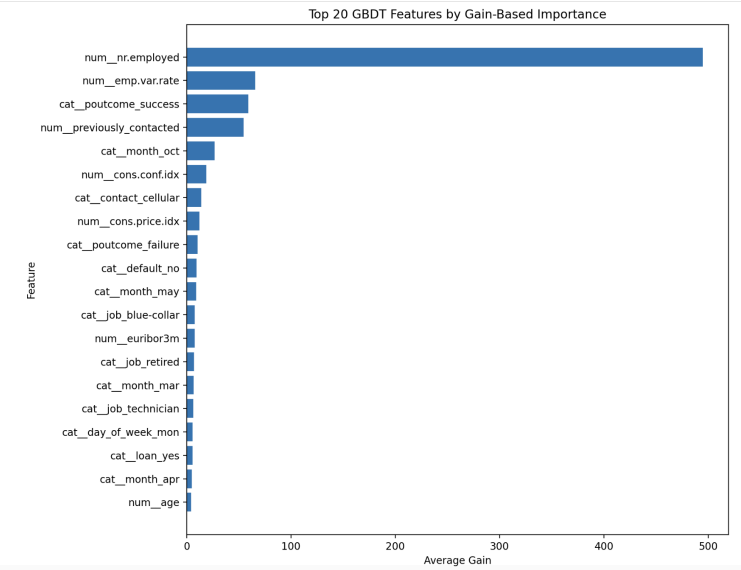
experiment string	learning_rate float64	max_depth int64	subsample float64	reg_lambda float64	best_iteration int64	validation_AP float64
subsample_1.0	0.1000	3	1.00	1.00	189	0.4937
max_depth_6	0.1000	6	0.8000	1.00	41	0.4920
learning_rate_0.1	0.1000	3	0.8000	1.00	254	0.4914
max_depth_4	0.1000	4	0.8000	1.00	118	0.4893
learning_rate_0.01	0.0100	3	0.8000	1.00	1019	0.4884
subsample_0.6	0.1000	3	0.6000	1.00	212	0.4879
learning_rate_0.3	0.3000	3	0.8000	1.00	39	0.4854
max_depth_2	0.1000	2	0.8000	1.00	288	0.4838

3.3 Training Behavior and Feature Importance

The training and validation log-loss curves decreased during the initial boosting iterations and then gradually leveled off. The selected model reached its best iteration at 189, indicating that additional boosting rounds did not improve validation log loss enough to keep training.



Gain-based feature importance was used to identify features that contributed most to the model's predictions. The most influential feature was nr.employed, followed by emp.var.rate, poutcome_success. These results suggest that employment-related economic indicators and previous campaign outcomes were particularly useful for predicting term deposit subscriptions.



4. Multi-Layer Perceptron

4.1 Model and Experimental Design

The MLP was implemented using Scikit-learn's MLPClassifier with the Adam optimization algorithm (solver="adam"). The ReLU activation function was used, and built-in early stopping was disabled. Training was limited to a maximum of 300 iterations. Three groups of hyperparameter experiments were conducted:

- Network architecture: (64,), (128, 64), and (128, 64, 32).

- Initial learning rate: 0.0003, 0.003, and 0.01.
- L2 regularization (alpha): 0.001, 0.1, 1.0, and 5.0.

Other settings were held fixed within each experiment group. Validation AP was used to compare configurations and select the final model.

4.2 Hyperparameter Results

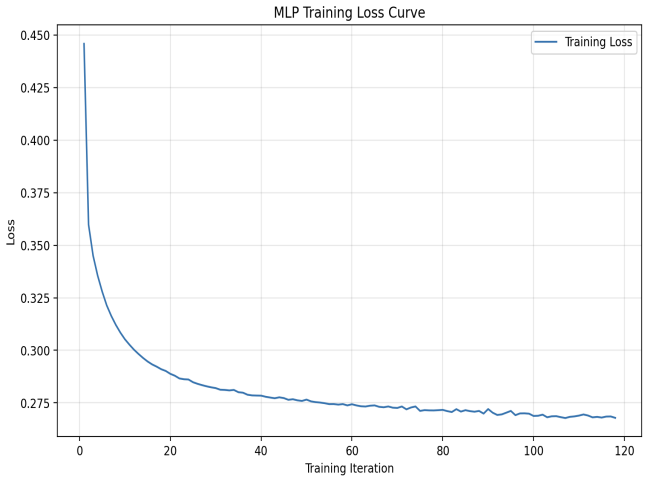
The best-performing MLP configuration used two hidden layers with 128 and 64 neurons, an initial learning rate of 0.001, and an L2 regularization parameter of 1.0.

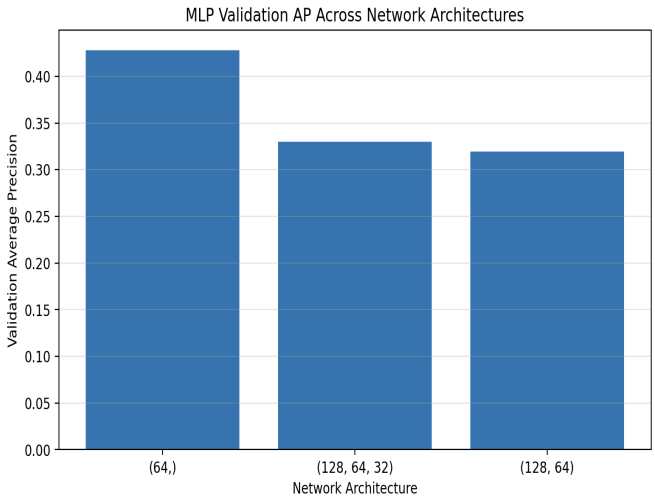
experiment string	hidden_layer_siz... mixed	learning_rate_init float64	alpha float64	n_iter int64	training_loss float64	validation_AP float64
alpha_1.0	(128, 64)	0.0010	1.00	118	0.2679	0.4777
alpha_5.0	(128, 64)	0.0010	5.00	74	0.2893	0.4645
architecture_(64,)	(64,)	0.0010	0.0001	300	0.2261	0.4279
learning_rate_0.00…	(128, 64)	0.0003	0.0001	300	0.1762	0.4001
alpha_0.1	(128, 64)	0.0010	0.1000	190	0.1639	0.3664
learning_rate_0.01	(128, 64)	0.0100	0.0001	131	0.0561	0.3302
architecture_(128,…	(128, 64, 32)	0.0010	0.0001	226	0.0610	0.3299
alpha_0.001	(128, 64)	0.0010	0.0010	300	0.0778	0.3255
architecture_(128,…	(128, 64)	0.0010	0.0001	300	0.0794	0.3196
learning_rate_0.003	(128, 64)	0.0030	0.0001	161	0.0572	0.3040

The results show that increasing network depth and width did not automatically improve performance. The single-hidden-layer architecture achieved a higher validation AP than the deeper architectures in the architecture experiments. However, the selected two-layer model achieved the highest overall validation AP when stronger L2 regularization was applied. The learning-rate experiments showed that the lowest tested learning rate achieved the highest AP among the three learning-rate configurations. The regularization experiments showed that increasing alpha from 0.001 to 1.0 substantially improved validation AP, while increasing it further to 5.0 resulted in a small decrease.

4.3 Convergence and Training Loss

Four of the ten MLP experiments produced convergence warnings, meaning they reached the maximum iteration limit without satisfying the optimizer's convergence criterion. The selected model, however, converged successfully after 118 iterations. The selected MLP's training loss decreased over successive iterations, indicating that the network learned from the training data.





5. Final Model Evaluation and Comparison

5.1 Model Selection

The final GBDT and MLP configurations were selected based on validation AP. After model selection was completed, the test set was used for final evaluation.

```
===== SELECTED GBDT MODEL =====
Configuration: subsample_1.0
Parameters: {'objective': 'binary:logistic',
Validation AP: 0.4936722678055503

===== SELECTED MLP MODEL =====
Configuration: alpha_1.0
Architecture: (128, 64)
Learning rate: 0.001
Alpha: 1.0
Validation AP: 0.477731740505429
```

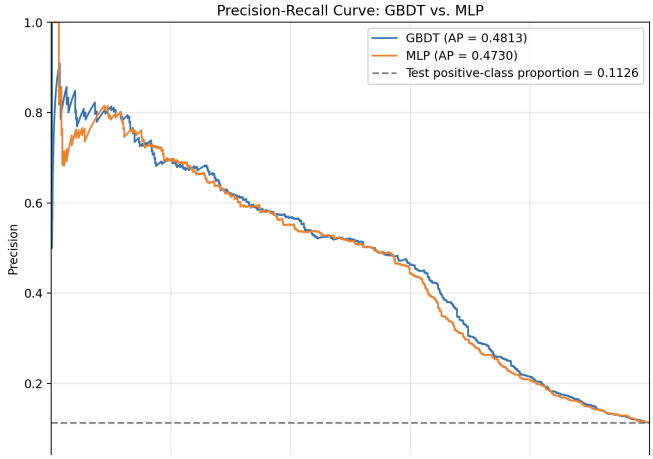
5.2 Test Performance

Both models were evaluated using accuracy, precision, recall, F1-score, and Average Precision. A classification threshold of 0.5 was used to calculate the threshold-dependent metrics, while AP was calculated using predicted probabilities. The GBDT achieved the higher test AP, indicating slightly better overall ranking performance. The MLP achieved marginally higher accuracy, recall, and F1-score, while the GBDT achieved higher precision. Both models had similar overall accuracy, but their recall values were relatively low. This indicates that at the 0.5 classification threshold, both models missed a substantial proportion of actual subscribers. Given the class imbalance, accuracy alone does not adequately describe model performance.

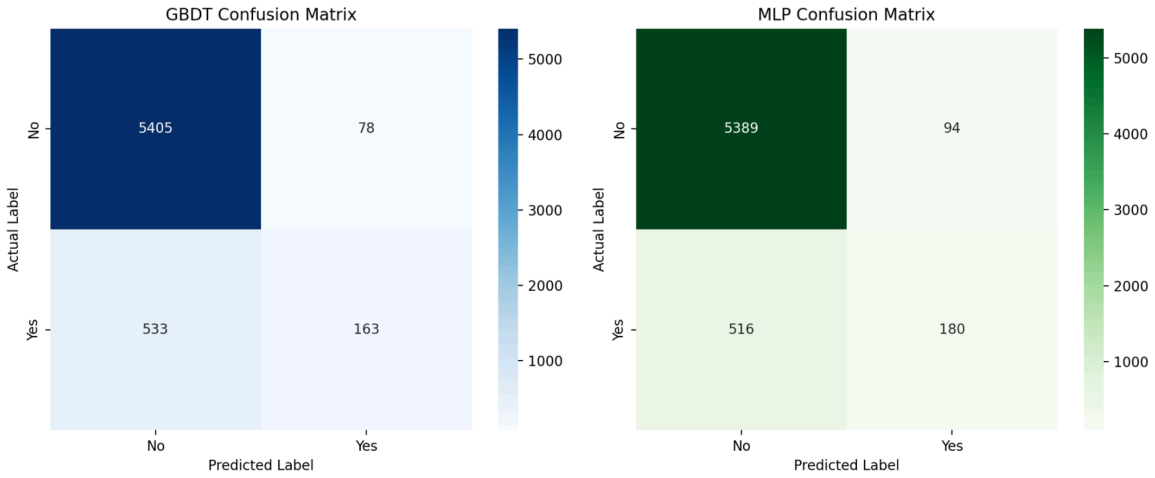
	Accuracy float64	Precision float64	Recall float64	F1 float64	AP float64
GBDT ...	0.9011	0.6763	0.2342	0.3479	0.4813
MLP	0.9013	0.6569	0.2586	0.3711	0.4730

5.3 Precision-Recall Curves and Confusion Matrices

The precision-recall curves provide a comparison of model performance across different classification thresholds. The horizontal reference line represents the positive-class proportion in the test set (11.264%), which corresponds to the expected AP of a random-ranking classifier. The GBDT achieved a slightly higher AP of 0.4813 compared with 0.4730 for the MLP. Both models performed substantially better than the random-ranking reference.



Confusion matrices were generated using a classification threshold of 0.5 to examine the numbers of true negatives, false positives, false negatives, and true positives. The GBDT correctly identified 5,405 non-subscribers and 163 subscribers, while incorrectly classifying 78 non-subscribers as subscribers and missing 533 actual subscribers. The confusion matrices illustrate that both models were much better at identifying non-subscribers than subscribers, reflecting the imbalanced target distribution.



5.4 Computational Efficiency

The GBDT trained substantially faster than the MLP. Although MLP preprocessing was slightly faster, its model fitting time was approximately 28 times longer than that of GBDT in this experiment.

	Model string	Preprocessing Ti... float64	Model Fit Time (... float64
0	GBDT (XGBoost)	0.1099	0.5373
1	MLP	0.0826	14.90

5.5 Discussion and Conclusion

GBDT achieved slightly higher test AP (0.4813) than MLP (0.4730), while MLP achieved slightly higher recall and F1-score at the 0.5 threshold. GBDT also trained substantially faster and was less sensitive to hyperparameter choices.Both models had relatively low recall, reflecting the challenge of predicting the minority positive class. Four MLP experiments did not converge within the 300-iteration limit, although the selected MLP converged successfully. Overall, GBDT performed slightly better in terms of ranking performance and computational efficiency, making it the stronger model for this dataset.

5.6 AI Tool Disclosure

I used AI tools to check and fix spelling and grammar in my report, look up API documentation, troubleshoot runtime errors, and review the assignment to make sure I had addressed all the requirements and didn't miss anything.

Colab link: https://colab.research.google.com/drive/16ysBzLgSpHjuqp8jd8ttVpFNje3NM6Fb

Github Repo: https://github.com/CharlotteZZh/AppliedML_hw2